

Partiel Module d'ALGORITHMIQUE & SDA L3-Info**NOTES**

- ♦ Les réponses aux questions doivent être claires et justifiées.
- ♦ Pour chaque algorithme (ou fonction en C), vous aurez soin d'expliquer les différentes actions de l'algorithme ainsi que le passage des paramètres.
- ♦ Chaque algorithme (ou fonction en C) supplémentaire utilisé doit être défini.

EXERCICE 1 (3.0 PTS)

- 1) Proposer en langage algorithmique (ou C), un algorithme où les trois cas de complexités sont différents (**Justifier votre réponse**).
- 2) Illustrer graphiquement le déroulement sur le tableau suivant : [3, 2, 6, 9, 1, 5, 8, 7, 11, 4] des algorithmes de tri suivant :
 - a) Tri Fusion
 - b) Tri Rapide

EXERCICE 2 (2.5 PTS)

- 1) En ne comptabilisant que les **multiplications (ligne I)**, trouver et analyser en fonction de n l'équation de la complexité en temps de la fonction suivante :

Fonction $tata(n) \rightarrow \text{Entier}$	Paramètres Formels
Début	
$som \leftarrow 0$	n : entier. Paramètre en entrée (E)
Pour i variant de 1 à n faire	
pour j variant 1 de i faire	
$som \leftarrow som + i * j$ (I)	Variables intermédiaires :
finpour	i, j, som : entier.
FPour	
Renvoyer som	
FIN	

- 2) En ne comptabilisant que les exécutions de l'affichage produit par la procédure **écrire (ligne I et II)**; trouver puis résoudre en fonction de n l'équation de la complexité de la fonction $tata$.

Procédure $tata(n)$ **DEBUT**

```

si n > 1 alors
    m ← n / 3
    pour i variant de 1 à n faire
        écrire('tata-tata') (I)
    FPour
        tata(m)
    ...
    tata(m)
    ...
    tata(m)
    ...

```

Sinonécrire('tata-tata') **(II)****FSi****FIN****Paramètres Formels :** n : Entier (E)**Variables intermédiaires :** m : Entier**EXERCICE 3 (11.25PTS) : LISTES CHAÎNÉES**

On souhaite calculer le PGCD d'un ensemble fini d'entiers donnés. Pour cela, on décompose chaque entier en un produit de facteurs premiers.

On suppose que les entiers sont tous distincts et supérieurs à 1. Les entiers sont stockés dans une **liste chaînée circulaire** d'entiers dont la structure est définie comme suit :

```

typedef struct cellule{
    int donnee;
    struct cellule *suivant ;
}*ListeEntier;

```

On suppose également que les nombres premiers sont stockées dans une liste linéaire simplement chaînée contenant suffisamment des nombres premiers :

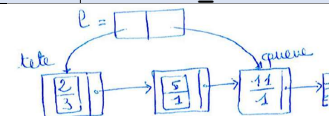


La décomposition d'un entier en facteurs premiers sera représentée par une liste linéaire simplement chaînée et **classé dans l'ordre croissant selon le facteur premier** de la décomposition. Pour faciliter la gestion de cette liste, on rajoutera deux adresses : **la première adresse pour indiquer la tête de la liste et la seconde adresse pour indiquer la queue de la liste**. La structure de la décomposition en facteur premier est définie comme suit :

<pre> typedef struct{ int fact ; // indique le facteur premier int nbOcc ; // nombre d'occurrences du facteur premier }FactPremier; </pre>	
<pre> typedef struct cellule{ FactPremier donnee; struct cellule *suivant ; }*ListeDFP; </pre>	<pre> typedef struct { ListeDFP tete; ListeDFP queue; }ListeTQ_DFP; </pre>

Exemple 1 :

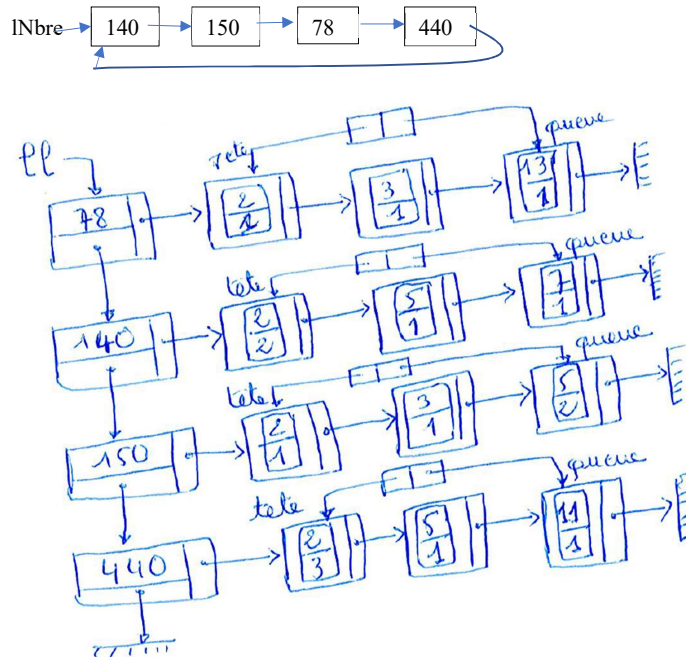
La décomposition de l'entier $440 = 2^3 * 5^1 * 11^1$ en facteurs premiers est représentée par la liste chaînée ci-contre :



- 1) **Ecrire en C**, l'insertion en queue d'un facteur premier (fat, occ) donné dans une liste de décomposition des facteurs premiers (ListeTQ_DFC).
- 2) Ecrire en langage algorithmique (ou en C), la décomposition d'un entier donné en facteurs premiers. Les facteurs premiers seront représentés par une liste chaînée (ListeTQ_DFC).
- 3) **Ecrire en C**, l'adresse du nombre premier p donné qui possède la plus grande occurrence dans une liste non vide de décomposition en facteurs premiers donnée (ListeTQ_DFC). Cette fonction retourne null si le nombre premier p n'existe pas dans la liste.
- 4) Ecrire en langage algorithmique (ou en C), un algorithme **itératif** d'intersection de deux listes chaînées représentant les facteurs premiers.
- 5) Ecrire en langage algorithmique (ou en C), un algorithme **récuratif** d'intersection de deux listes chaînées représentant les facteurs premiers.

Par suite, la liste circulaire d'entiers (INbre) sera transformée en une **liste triée de listes (II)** des facteurs premiers selon l'exemple suivante :

Exemple 2 :



- 6) **Ecrire en C**, la structure de données représentant la liste de listes des facteurs premiers selon l'exemple 2.
- 7) **Ecrire en C**, l'insertion triée d'un entier donné (nb) avec sa décomposition en facteurs premiers (ldfp) dans une **liste triée de listes (II)** des facteurs premiers.
- 8) En utilisant la liste des nombres premiers (IPre) ; écrire en langage algorithmique (ou en C), la construction de la **liste triée de listes (II)** des facteurs premiers à partir d'une liste circulaire d'entiers donnée (INbre).
- 9) En utilisant la liste des nombres premiers (IPre) ; écrire en langage algorithmique (ou en C), le calcul du PGCD d'une suite d'entiers donné sous forme d'une liste circulaire (INbre).

EXERCICE 4 (3.25 PTS) : LISTES CHAÎNÉES CIRCULAIRES AVEC SENTINELLE

- 1) **Sans la création de nouvelles cellules** sauf l'allocation mémoire de la cellule sentinelle ; **écrire en C**, un algorithme **itératif** qui transforme une liste d'entiers circulaire avec sentinelle en une autre **liste triée** d'entiers circulaire avec sentinelle.
- 2) Réécrire **récurivement** en C, l'algorithme précédente.

On suppose connu les primitives et les algorithmes suivants :

Structure ListeEntier

Fonction donneeLE : Liste → TElement
 Fonction suivantLE : ListeEntier → ListeEntier
 Fonction initLE : [] → Liste
 Fonction estVideLE : Liste → Booléenne
 Fonction allocMemRempleLE : Entier → ListeEntier

Structure FactPremier

Fonction fact : FactPremier → Entier
 Fonction nbOcc : FactPremier → Entier

Structure ListeDFP

Fonction donneeLDFP : ListeDFP → TElement
 Fonction suivantLDFP : ListeDFP → Liste
 Fonction initLDFP : [] → ListeDFP
 Fonction estVideLDFP : ListeDFP → Booléenne
 Fonction allocMemRempleLDFP : FactPremier → ListeDFP

Structure ListeTQ_DFC

Fonction tete : ListeTQ_DFC → ListeDFP
 Fonction queue : ListeTQ_DFC → ListeDFP
 Fonction initLDFP_TQ : [] → ListeTQ_DFC
 Fonction estVideLDFP_TQ : ListeTQ_DFC → Booléenne